

Roteiro de Apresentação — Árvore B em Memória Secundária

Duração alvo: 20 minutos · 10 slides · AED-PG-2026

Como usar: cada slide tem (tempo), a **mensagem-chave** (o que a banca tem que levar) e a **fala** (texto natural, pode adaptar). As frases em *itálico* são transições para o próximo slide. No fim há um bloco de **perguntas prováveis**.

Orçamento de tempo (total ≈ 19 min + 1 min de folga):

Slide	Assunto	Tempo
1	Capa / abertura	1:00
2	Decisões de implementação	2:30
3	Métodos implementados	2:00
4	Experimentos	2:00
5	Métricas	1:30
6	Resultados: impacto de m	3:00
7	Resultados: reaproveitamento	2:00
8	Uso de LLM + 2 máquinas	1:30
9	Dificuldades / vantagens	1:30
10	Aplicações / conclusão	2:00

Slide 1 — Capa 1:00

Mensagem-chave: o que é o trabalho em uma frase.

Fala: "Bom dia/boa tarde. Nosso trabalho é a implementação de uma **Árvore B de ordem m que opera estritamente em memória secundária** — ou seja, a árvore vive inteira em um arquivo binário no disco, e a regra de ouro é: **toda operação lê ou escreve um nó por vez**, nunca carregando a árvore para a RAM. Eu sou o Cássio, e ao longo da apresentação vou mostrar a estrutura, os métodos, e principalmente a **avaliação experimental**, que é onde estão os resultados mais interessantes."

"Vamos começar pelas decisões de implementação."

Slide 2 — Decisões de implementação 2:30

Mensagem-chave: como garantimos "1 nó por I/O" e onde fica cada coisa no arquivo.

Fala: "Quatro decisões definem o projeto. Primeiro, a **ordem m é uma constante de compilação** — a estrutura é totalmente parametrizada, então a mesma classe roda como ordem 3 ou ordem 1000 só mudando um `#define`. Segundo, **a raiz fica no header do arquivo**, no registro 0, junto com o total de nós e a cabeça da lista de livres. Assim, ao abrir o arquivo, sabemos onde começar sem varrer nada. Terceiro, e mais importante para o enunciado: **um nó por I/O**. Cada nó é um registro de tamanho fixo (`PAGE_SIZE`); `readNode` e `writeNode` leem/escrevem exatamente um registro. A árvore **nunca** é carregada inteira — isso é o que o enunciado exige e o que desqualifica trabalhos que sobem tudo para a RAM. Quarto, o **reaproveitamento de nós**: quando uma remoção libera um nó, ele entra numa **lista de livres encadeada dentro do próprio arquivo** — o nó livre é marcado com `n = -1` e guarda o ponteiro para o próximo. Na próxima alocação, reusamos esse nó antes de crescer o arquivo. À direita vocês veem uma árvore de ordem 3 de verdade, exportada pelo nosso

código via Graphviz — repare que cada caixa mostra o **RRN**, o número do registro no disco."

"Esses são os métodos que implementamos sobre essa base."

Slide 3 — Métodos implementados 2:00

Mensagem-chave: cobrimos busca, inserção e remoção completas + ferramentas.

Fala: "As três operações essenciais estão completas. A **busca** é m-way: em cada nó as chaves particionam o universo em faixas, e descemos pela faixa certa até a folha. A **inserção** é bottom-up: acha a folha, insere, e se o nó estoura, faz o **split**

- a mediana sobe para o pai, e isso pode propagar até criar uma nova raiz. A **remoção** é a parte mais delicada: se a chave está num nó interno, trocamos pelo

sucessor in-order; e se um nó fica abaixo do mínimo, reparamos com

redistribuição (pega emprestada uma chave do irmão) ou **fusão** com o irmão. Além disso implementamos as **funcionalidades de monitoramento** que o enunciado incentiva: impressão hierárquica, cálculo de altura, exportação para Graphviz, e - fundamental - o **contador de acessos ao disco**, que é a métrica central da avaliação."

"Com a classe pronta, partimos para os experimentos."

Slide 4 — Experimentos 2:00

Mensagem-chave: a matriz de testes é ampla e cobre todos os itens do enunciado.

Fala: "Fizemos quatro experimentos. O **Experimento 1** varia a ordem m de 3 até 1000, com N fixo em cem mil chaves - é o que mostra o impacto de m. O **Experimento 2** varia o tamanho do conjunto de mil a um **milhão** de chaves, para ordens pequena e grandes. O **Experimento 3** mede a ocupação do arquivo **com e sem** reaproveitamento. E o **Experimento 4** separa o tempo de CPU do tempo de espera de I/O. Tudo isso nos modos **aleatório e sequencial**. E como novidade, rodamos a suíte em **duas máquinas** - meu notebook e o servidor Titan da USP - para validação cruzada. O gráfico mostra o ponto central que vamos detalhar: o número de acessos por busca quase **não cresce** mesmo multiplicando N por mil, porque a árvore é logarítmica na base m."

"Antes dos números, as métricas que medimos."

Slide 5 — Métricas 1:30

Mensagem-chave: a métrica honesta é o contador de acessos, não o relógio.

Fala: "Medimos seis coisas. A principal é **acessos ao disco** - o contador de leituras e escritas de nós, total e média por operação. Essa é a métrica **honesta e independente de hardware**. Medimos também a **altura**, a **ocupação do arquivo** (nós físicos, nós livres e bytes), e o **tempo**, decomposto em CPU de usuário, CPU de sistema e espera de I/O, usando getrusage. Uma observação importante que aparece já no gráfico de altura: a altura cai em degraus conforme m cresce - de 14 níveis em m=3 para 2 níveis em m grande."

"Agora os resultados - começando pelo impacto da ordem m."

Slide 6 — Resultados: impacto de m 3:00 (*) (slide mais importante)

Mensagem-chave: I/O despenca com m e SATURA - esse é o achado principal.

Fala: "Esse é o resultado central. Olhem a coluna **busca I/O por operação**: com **m=3**, cada busca custa **13 acessos** ao disco e a árvore tem **14 níveis**. À medida que aumentamos m, isso despenca: m=16 já cai para ~5 acessos, e a partir de **m≈512** a busca custa só **2 acessos**, com altura 2. Mas reparem no **ponto-chave**: o ganho **satura**. Entre m=64 e m=128 a busca já está em ~3 acessos e quase não melhora mais. Por quê? Porque a árvore já tem só 2-3 níveis - não dá para ficar mais rasa. A partir daí, aumentar m **não reduz mais I/O**; só aumenta o custo de **CPU** para varrer as chaves dentro do nó. Ou seja: existe um **m ótimo** - grande o suficiente para a árvore ser rasa, mas não exagerado. No mundo real, esse m é escolhido para o nó caber em um bloco de disco. É exatamente o comportamento que a teoria prevê, e conseguimos medi-lo empiricamente."

"O segundo resultado é sobre espaço: o reaproveitamento de nós."

Slide 7 — Resultados: reaproveitamento 2:00

Mensagem-chave: a free list economiza ~27-30% de espaço.

Fala: "Aqui medimos a **ocupação do arquivo** num cenário de *churn*: inserimos N chaves, removemos metade, e re inserimos outra metade. Com o reaproveitamento **ligado**, a reinserção consome os nós que ficaram livres e o arquivo **não cresce**. Com ele **desligado**, cada alocação acrescenta um registro novo no fim e o arquivo **incha**. O resultado é consistente em todas as ordens: o reaproveitamento **economiza entre 27 e 30%** do tamanho do arquivo. Para uma estrutura de disco, onde espaço e localidade importam, isso é significativo."

"Sobre o processo: usamos um LLM e validamos em duas máquinas."

Slide 8 — Uso de LLM + duas máquinas 1:30

Mensagem-chave: transparência sobre IA + prova de determinismo.

Fala: "Por transparência: usamos o **Claude**, via Claude Code, como apoio - para discutir os pseudocódigos, **encontrar dois bugs** (um de header desatualizado na inserção e um de eofbit do fstream), e para gerar o harness de testes e os gráficos. **Tudo o código foi revisado e validado manualmente**. E o gráfico mostra a validação em duas máquinas: notebook contra o servidor Titan. Os **tempos** diferem, claro - hardwares diferentes - mas **todas as métricas de acesso a disco saíram idênticas**, byte a byte. Isso **prova** que a implementação é determinística e portátil; a diferença é só de máquina, não de comportamento."

"Quais foram as dificuldades e os trade-offs?"

Slide 9 — Dificuldades · Vantagens × Desvantagens 1:30

Mensagem-chave: entendemos o custo-benefício da estrutura.

Fala: "As maiores dificuldades foram a **remoção** - manter o caminho na ordem certa para reparar o underflow - e detalhes de I/O como o eofbit, que fazia leituras falharem silenciosamente. Sobre os trade-offs: a Árvore B é **imbatível para disco** - rasa, balanceada, pouco I/O. Em troca, os nós podem ficar **subocupados** (no caso sequencial, ~50% cheios), a remoção é complexa, e como vimos, m muito grande troca I/O por CPU. É um **equilíbrio** consciente."

"Para fechar: onde isso é usado e as conclusões."

Slide 10 — Aplicações · Conclusão · Referências 2:00

Mensagem-chave: isso roda no mundo todo + recapitular os 3 achados.

Fala: "Onde a Árvore B aparece na prática? Em praticamente **todo banco de dados** - os índices do MySQL/InnoDB e do PostgreSQL são B-trees (na verdade B+ trees); em

sistemas de arquivos como NTFS e ext4; e em key-value stores. Recapitulando nossas três conclusões: um - o I/O por operação **cai com m até saturar**, por volta de m de 64 a 128; dois - o reaproveitamento de nós **economiza ~30%** de espaço; três - os resultados são **determinísticos**, validados em duas máquinas. As referências estão no slide, com destaque para o artigo clássico do Comer, 'The Ubiquitous B-Tree'. Obrigado! Fico à disposição para perguntas."

Perguntas prováveis (preparação)

"Árvore B não é binária? Ordem 3 não seria o padrão?"

→ "Não. Binária é a BST, com 2 filhos. A Árvore B é **m-way**: até m filhos. O 'B' não é de *binary* - é de balanceada (Bayer). O objetivo é justamente **não** ser binária: m grande deixa a árvore rasa e reduz acessos a disco. Aliás, ordem 2 (que seria 'binária') é degenerada e nosso código a proíbe; ordem 3 é só o **menor caso** que testamos, o pior em número de acessos."

"Por que o tempo aparece quase todo como CPU, e não como espera de I/O?"

→ "Porque o writeNode faz flush para o **cache de páginas do SO**, não um fsync no disco físico. Numa máquina com RAM sobrando, o 'I/O' vira CPU de sistema. Por isso a métrica honesta e comparável é o **contador de acessos**, não o relógio."

"Onde fica a raiz e por que isso importa?"

→ "No header, registro 0. Assim abrimos o arquivo e já sabemos a raiz com uma leitura de metadado, sem custo de busca."

"Como funciona o reaproveitamento exatamente?"

→ "Lista de livres encadeada no próprio arquivo: a cabeça fica no header (free_head), e cada nó livre guarda o próximo em K[1], marcado com n = -1. allocNode puxa dessa lista antes de estender o arquivo."

"Qual o m ideal na prática?"

→ "Aquele em que o nó preenche um **bloco de disco** (ex.: 4 KB ou 8 KB). Aí cada acesso a disco traz o máximo de chaves úteis - é onde a curva do slide 6 satura."

"Por que a inserção sequencial gera arquivo maior?"

→ "Porque o split sempre acontece à direita, deixando os nós ~50% cheios; no aleatório a ocupação fica ~69% ($\ln 2$), então cabem mais chaves por nó."

Dicas de cronometragem

- Se estiver **adiantado**: detalhe mais o slide 6 (o ponto de saturação) e o 7.
- Se estiver **atrasado**: nos slides 3 e 5, leia só os títulos dos bullets.
- **Não leia os slides palavra por palavra** - eles são o apoio visual; a fala acima é a narração.